

Machine Learning Based Anomalous Network Traffic Detection

Jason Moss, Daniel Way, Sydney Marshall

Abstract—As network intrusion attacks become more prevalent, many businesses and other institutions are turning to machine learning in order to train their intrusion detection systems. Many of these groups are using various algorithms in order to create an efficient IDS. In this paper, we used the publicly available network intrusion dataset NSL-KDD to train and test the isolation forest, autoencoder, and XGBoost algorithms. We experiment with different preprocessing methods and then tune hyperparameters in order to achieve greater performance. After tuning the algorithms, we analyze their performance when it comes to detecting anomalous network data in order to select the best method for using NSL-KDD to create an accurate IDS.

Index Terms—Machine Learning, Anomaly Detection, Cybersecurity.

I. INTRODUCTION

How dangerous would it be if the transportation industry and their technologies like traffic lights, sensor networks, vehicle-to-infrastructure communication, and websites did not implement intrusion detection systems (IDS)? Vulnerabilities, anomalies, and other network intrusion attacks could spread throughout and create significant damage to public safety. All of those Cybersecurity risks are becoming increasingly more prevalent and change lives for the worse. These include but are not limited to denial of service (DoS), data spoofing, and signal manipulation. The systems that the transportation industry needs 24/7 real-time and accurate information to properly conduct business.

DoS attacks have been discovered as one of the most common attack vectors that the transportation industry sees. A DoS attack is an umbrella term for the different ways an authentic user can run into issues with accessing systems, devices, or resources due to malicious threat actors. As mentioned before, it disrupts the continuous and secure data exchanges that the transportation industry depends on. Imagine not being able to access a carrier’s online ticketing system, resulting in customers not being able to buy tickets or check schedules/information from purchased tickets. This would be very frustrating for customers, they might be stuck in an airport, or an unfamiliar destination, and miss flights by not knowing correct times or gates. On the other side, businesses and airlines will be affected by having reputational damage, losing customers, and having an overall difficult time with managing incidents. Another instance within the transportation industry this has affected are truck trailers. An attacker used an antenna to exploit a vulnerability relating to diagnostic messages on a truck’s Anti-lock Braking System (ABS). They can force the brakes to lock or impair them completely. This

is intensely dangerous to the driver of the truck and others on the road.

The next kind of attack that is prevalent is defacement. Defacement is when malicious actors penetrate an online platform or website and change the content or how it looks. They will exploit vulnerabilities and gain administrative access. For example with another ticketing system, they could redirect users that are trying to visit that page and redirect them to their own defaced page. From there, they can capture their credentials, payment, flight and personal information. In all, there are many attacks out there that can cause a loss or credibility, interrupt service, and introduce data breach risks.

Our goal was to assess machine learning algorithms so they can better detect and recognize patterns that are pertinent to transportation networks. We found that the publicly available data set, NSL-KDD, would be the best benchmark dataset for our project. This dataset is an improved version of the dataset KDD’99. There were found to be multiple issues with the original dataset. It had redundant records, distorted metrics, vague attack definitions, and a disproportionate amount of attack types. The solution was to adapt to a new version that no longer has redundant records in the training and duplicate entries in test sets, more balanced difficulty levels and manageably sized. After implementing these proposed changes, NSL-KDD had advantages over KDD’99 such as, less bias, expanded performance variation and accuracy, and a more convenient way of experimenting without sampling.

Utilizing an IDS is essential for mitigating and identifying the earlier mentioned threats. That is why we implemented three anomaly detection models, Isolation Forest, Autoencoder, and eXtreme Gradient Boosting (XGBoost). We found these to be the best due to their strengths in unsupervised and supervised learning. “Supervised learning is a machine learning technique that uses labeled data sets to train artificial intelligence (AI) models to identify the underlying patterns and relationships.” For each example that is a part of the training dataset, the algorithm knows what the correct output should be or what data is “normal.” This allows the model to measure its accuracy and learn over time. From there, it attempts to conclude new examples that it has not seen before. Unsupervised is the opposite, it uses unlabeled data sets. Unsupervised does not know prior, what the correct output should be. It learns patterns straight from the input data. XGBoost is supervised and Autoencoder and Isolation Forest are unsupervised.

II. LITERATURE REVIEW

There are many researchers who have done similar work to us. They have experimented with different machine learning techniques on the NSL-KDD dataset in order to figure out what setup leads the the best model to detect anomalous network traffic data. The authors in [1] experiments with a novel methodology that combines isolation forest and one class support vector machine(OCSVM) in order to create a model that outperforms other techniques as well as can more accurately detect threats without being aware of them. The authors compare their model to models done in other work to show that their model that combines isolation forest and OCSVM outperforms the other models it was tested against. The authors in [2] use three different machine learning methods with NSL-KDD for detecting anomalous network traffic. The three methods the authors use are support vector machine(SVM), logistic regression, and K-nearest neighbors(KNN). After selecting the nine best features with random forest, the authors set up and test the three methods. The support vector machine method achieves 87.58% accuracy, 86.91% precision, 87.31% recall, and 87.07% f1 score. The logistic regression method achieved 88.86% precision, 87.83 % precision, 87.98% recall, and 88.48% f1 score. The K nearest neighbors method achieved 98.24% accuracy, 97.99% precision, 97.91% recall, and 98% f1 score. The authors in [3] train an isolation forest model on the NSL-KDD dataset in order to detect network anomalies. The authors use OCSVM in order to help their model detect anomalies in an imbalanced dataset[3]. Their model is is trained on unknown data after splitting the test and train sets[3]. Their model achieved an AUC score of 98.3%.

[2] and [3] both perform feature selection on the NSL-KDD dataset. [2] selects the top nine features from NSL-KDD. [3] would select a varying number of features after training the data on all features in order to try and achieve a better accuracy with the selected features. We believe that selecting only nine features like [2] is not enough. We believe that nine features is too small for use on the NSL-KDD. We decided to try fifteen instead because we believe that is an appropriate number of features to select to properly train and test the NSL-KDD dataset.

III. METHODOLOGY

A. Preprocessing

The first step we took in our preprocessing was to import the libraries we needed. This would change over time as we realized we needed other libraries for certain parts of setting up our models. As we began our current version of our preprocessing, we used the `df_shrink` command to lower memory usage on our train and test datasets. We then added six features that our test dataset was missing from our training set. The six features we added to our testing set from our training set were `aol`, `harvest`, `http_2784`, `http_8001`, `red_i`, and `urh_i`. We then used label encoding on our training and testing datasets to convert our categorical data to numerical data. The columns we performed label encoding on were `classification`, `protocol_type`, `service`, `flag`, `land`, `logged_in`, `is_host_login`, and `is_guest_login`. The class column was transformed into

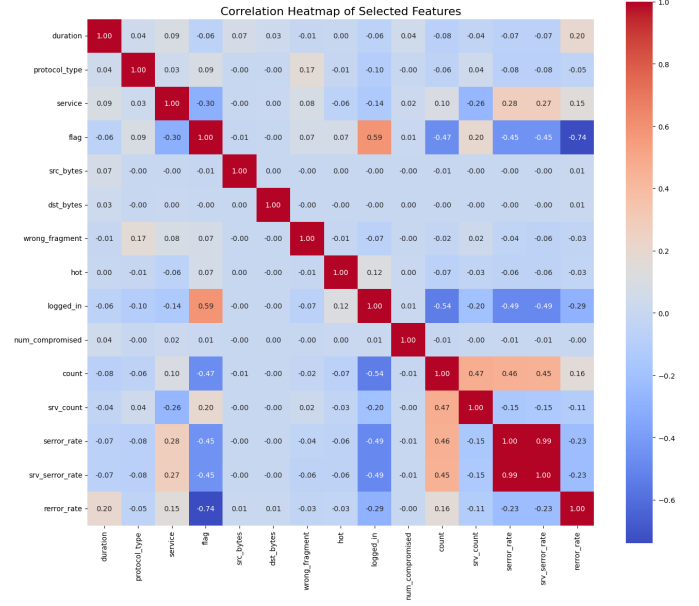


Fig. 1. Top 15 Features

classification and the old class column was dropped from the train and test datasets. We then set a variable to contain the classification column of the training dataset. We then selected the top fifteen features using `SelectKBest` as well as the classification columns to be used in both datasets.

The top fifteen features as shown in Fig. 1 were `duration`, `protocol_type`, `service`, `flag`, `src_bytes`, `dst_bytes`, `wrong_fragment`, `hot`, `logged_in`, `num_compromised`, `count`, `srv_count`, `error_rate`, `srv_error_rate`, and `rerror_rate`.

We then perform one-hot encoding on all nominal categories except the classification column. The nominal categories we perform one-hot encoding on are `protocol_type`, `service`, `flag`, and `logged_in`.

We then copy the test dataset's classification column into a variable called `target_test`. Following this, we record the train dataset's columns into `column_names` for use when converting back to dataframes. After this, we perform robust scaling on both datasets and convert them back to dataframes.

An earlier version of our preprocessing was somewhat similar to what we have now. We did not select the top 15 features or drop the class column. First, we added the missing features to our train set after using `df_shrink`. Second, we performed one hot encoding on all nominal categories except class. The difference here is that that we did not perform feature selection so we encoded the features `protocol_type`, `service`, `flag`, `land`, `logged_in`, `is_host_login`, and `is_guest_login`. Third, we performed label encoding on only the class column in the train and test datasets. Once we label encoded the class column into the classification column, we dropped the class column from both datasets. We then performed robust scaling on all columns in the test and train datasets. Our final step was to convert our sets back to dataframes so they could be used in the isolation forest model.



Fig. 2. Isolation Forest Setup

B. Isolation Forest

After preprocessing, we implement PCA to reduce the amount of data for faster performance. We use forty-four components for PCA. After implementing PCA, we set up the isolation model and pass in the parameters. Once the parameters are passed into the model, we run a loop that changes the value -1 in the model's prediction to 0 so that we can properly compare it to our variable `target_test`. Once the loop is done running we figure out the accuracy, precision, f1, and auc metrics by comparing the variables `target_test` and `y_test_pred`. The variable `target_test` contains the actual data that was in the classification column of the test dataset. The `y_test_pred` variable contains the isolation forest model's predictions. The various metrics are calculated by passing `target_test` and `y_test_pred` into the functions `accuracy_score`, `metrics.precision_score`, `metrics.recall_score`, `metrics.f1_score`, and `metrics.roc_auc_score`. The metrics part in the last three function calls is the name we use to refer to the Scikit-learn library when we imported it. We then set up a confusion matrix to show the true positive, true negative, false positive, and false negative percentages. After setting up PCA and the isolation forest, we experimented with various parameters in order to achieve the best metrics. We ran loops that went through thousands of different values for each variable. Once we would achieve an increase in our accuracy and other metrics, we move on to next variable and run more over thousands of values and go with the instance where we has the highest increase in our metrics. We would repeat this process until it seemed that we could not increase our metrics anymore. Our estimator value is set to seventeen, our contamination values is set to 0.4309778452019331906885923, the number of samples we use for the training is eighty nine, and our state is set to 6780.

Fig. 2 shows our setup for the isolation forest. We take the

NSL-KDD dataset and perform preprocessing on it. After this we move on to the feature selection phase we determine what the most important features in NSL-KDD are. After we select the most important features in our dataset during the feature selection phase, we move to the initialization where we set up the isolation forest model and run the training dataset. Once we have finished the initialization phase, we move on to the anomaly scoring phase where we run our test set and view the results. When then move to the final phase where we look at the results of the anomaly scoring and determine what can be improved. If improvements can be made, we make the changed and start at the initialization phase again.

In earlier versions of our work, we were setting up the isolation forest without PCA and would just pass in our two variables and loop through different values. We tried to increase all the variables after each loop to see if we could get better metrics. We would experiment with changing specific variables such as only changing two after each loop to see if we could get better metrics. Eventually, we settled on looping through one variable at a time.

C. Local Outlier Factor

After preprocessing, the Train and Test data were both separated from their classification columns. Train was split into `X_train`, containing all the data minus the last column, and `y_train`, containing just the last classification column. The same thing happened with test but split into `X_test` and `y_test`.

Next, the Local Outlier Factor (LOF) algorithm was employed with $k=3$ nearest neighbors, a contamination ratio of 0.00522, and novelty detection enabled to identify previously unseen anomalies. The model was trained on the train data, and then subsequently used to classify the test instances. The raw LOF predictions were then converted from the default format, -1 = anomaly and 1 = normal, to align with the dataset convention of 0 = anomaly and 1 = normal.

Finally, the model performance was evaluated using accuracy and a detailed classification report, which included precision, recall, and F1-score. In addition to the standard classification metrics, model performance was further evaluated using a confusion matrix and a Receiver Operating Characteristic (ROC) curve.

D. Autoencoder

Once preprocessing was finished, we still needed to take another step in order to set the data up for our autoencoder model. We made a training dataset where we removed the anomalous data and only kept the normal data. This allows our model to recognize what normal data looks like and view differences to this as anomalous. After we make this change to our training dataset, we perform min-max scaling on both our training dataset and our test dataset. We perform min-max scaling because neural networks perform better when our data is on the same scale. We then transform our data back to dataframes and begin to setup our model. After setting our seed to we can reproduce our results, we set up seven layers for the autoencoder. The input layer is passed the shape

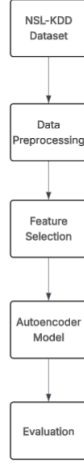


Fig. 3. Autoencoder Setup

of our scaled training data. We then reduce the dimensions through two encoding layers. We then take the hidden layer and pass it into the two decoding layers. Once the data passes through the two decoding layers, it is finally passed into the output layer to be reconstructed with the data's original dimensions. The activation function on the layers is set to RELU except for the output layer. The output layer uses Sigmoid. Once we are finished setting up our model, we compile our autoencoder. We set the optimizer to adam since it works well with large datasets. We use the mean squared error for our loss calculation. Finally, we use the `keras.metrics` to evaluate our model. We then pass our train data into our model and set our parameters. We set our epochs to 15, set shuffle to true, set our batch size to 798, set our seed to 939, and use our scaled training data to validate itself. Once we have passed our training dataset into the autoencoder, we then pass our testing dataset into the model. We calculate the reconstruction error by taking the mean of the difference of the true test data and the predictions determined by the autoencoder. We then pass the test set and the predictions into the SciKit `mean_squared_error` function to calculate the mean squared error. We then create a dataframe called `df_error` to the reconstruction error for our test set. We then create a list called `outliers` and place every entry where the reconstruction error is greater than the mean squared error into it. We then run a for loop that changes every entry in `y_test` to 1. We then run another for loop that changed every entry in `y_test` that is in the `outliers` list to 0. We then calculated our metrics by passing our `target_test` and `y_test` into the `accuracy`, `precision`, `recall`, `f1`, and `auc` functions. We also plotted our loss and validation loss, plotted an roc curve, and set up a confusion matrix. The setup of the autoencoder can be seen in Fig.3.

E. XGBoost

After our main preprocessing, we still needed to prepare the dataset for use in XGBoost for model training, to address class imbalance and to train an XGBoost classifier for anomaly detection. First, we copied the last columns from train and test

dataframes into `y_train_orig` and `x_train_orig`. Following this, we standardized the labels so that anomalous and normal samples are represented by zero and one respectively. `X_train` and `X_test` features were then separated from the target columns and combined into datasets `X_combined` and `y_combined`. This was to ensure consistent preprocessing and distribution across both sets.

A stratified train-test split was done to preserve the class distribution between normal and anomalous samples, using a 70/30 ratio with a random state of forty-two. To handle instances where one class may significantly outnumber the other, the `scale_pos_weight` perimeter was computed as the ratio of normal to anomalous samples in the training data. Weighing this helped the model pay equal attention to both classes during training.

Afterward, an XGBoost classifier was then initialized with the following hyperparameters: 200 estimators, a maximum tree depth of 6, and a learning rate of 0.1. Next, subsampling of 0.8 and column sampling of 0.8 was applied in order to improve generalization and prevent overfitting as much as possible. All of this was done with a `random_state` of 42.

Finally, the model was trained on the training set while being evaluated on the test set. Predictions were then completed, representing the final classification outputs of the trained model for normal vs anomalous traffic.

Once the model was completed, we calculated XGBoost's Receiver Operating Characteristic Area Under the Curve (ROC AUC) and generated a classification report. This was completed by passing `y_test_mapped` and `y_pred` into the `roc_auc_score` function that we imported from the `sklearn.metrics` library. Continuing onto the classification report, we generated it by passing `y_test_mapped` and `y_pred` into `classification_report` along with target names of "Anomaly (0)" and "Normal (1)" for better readability.

UPDATE DIAGRAM

In Fig. 4, we show our setup for the XGBoost model. We began by using the NSL-KDD dataset and performing preprocessing to prepare the data. After preprocessing, we moved on to feature encoding and normalization to ensure that all categorical and numerical values are in a format that we could actually use for model training. Once the data was properly encoded and normalized, we initialized and trained the XGBoost model, which used decision trees to classify the data. After training, we moved to the optimization phase where hyperparameters were fine tuned to improve accuracy and reduce overfitting. Finally, we analyzed the output and feature importance to evaluate which features had the greatest impact on the model's decisions and overall performance.

Originally when setting up XGBoost, we could not get a score of much higher than about 0.76, even after changing the hyperparameters. We were able to finally raise it minimally after implementing label encoding and one-hot encoding into preprocessing. However, after setting up the stratified split, our accuracy score jumped significantly. Although this did cause some slight overfitting, we were able to fix it after experimenting with the hyperparameters further.

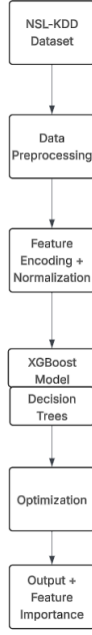


Fig. 4. XGBoost Set – UPDATE DIAGRAM

IV. RESULTS

Before we discuss the results of each of our models, we want to explain what the metrics we are discussing are. The first metric is accuracy. Accuracy shows how often a model is correct. The second metric is precision. Precision shows the percentage of true positives. The third metric is recall. Recall show the percentage of positive predictions. The fourth metrics is the F1 score. The F1 score shows the combined percentage of precision and recall. The final metric we discuss is the auc score. The auc score is the percentage that the model will make correct predictions.

A. Isolation Forest

Our current setup for isolation forest has achieved fairly high metrics. Our model has reached 91% accuracy, 92% precision, 86% recall, 89% f1 score, and and a 90% auc score. A graph showing our auc score can be seen in Fig.5

Our current confusion matrix can be seen in Fig.[6]. 94.8% of the normal data was correctly labeled as normal, 86.1% of anomalous data was correctly labeled as anomalous, 13.9% of anomalous data was incorrectly labeled as normal, and 5.2% of normal data was wrongfully labeled as anomalous. The confusion matrix seems to indicate that our isolation forest model struggles a bit more when classifying anomalous data. We had to run loops after implementing our current preprocessing in order to reach these metrics. We slowly increased through the loops until we reached our current metrics.

Our earlier version of the isolation forest model was not able to get to 90% accuracy. After running loops trying to get high metrics including an accuracy of 90% or higher, we were not able to get an accuracy of 90% or higher. The highest accuracy metric we could get in this earlier version was 89%.

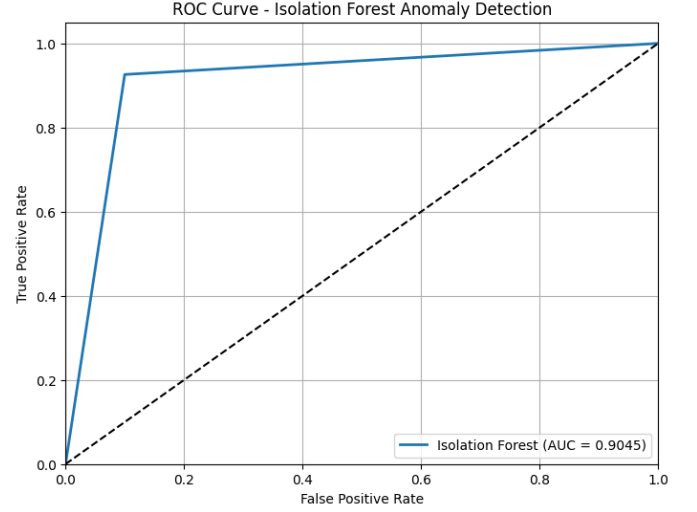


Fig. 5. Isolation Forest ROC Curve

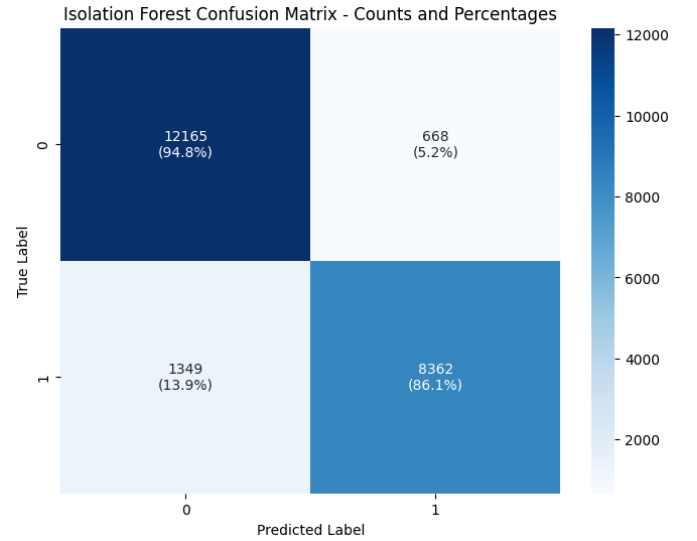


Fig. 6. Isolation Forest Confusion Matrix

We tried running loops to go through several different values to try and push the accuracy to 90% or higher. We were not successful at this and had to think of another way to try to push the accuracy to 90% or higher. This would eventually lead us to changing how the data was preprocessed as well as how many components we used with PCA.

B. Local Outlier Factor

The current LOF model has achieved an overall classification accuracy of 0.7688, indicating that approximately 76.9% of all test samples were correctly identified as either normal or an anomaly. This shows that the LOF algorithm was able to mostly distinguish between the normal and abnormal data patterns. However, this does show that many misclassifications still occurred.

A breakdown of model performance is presented in the classification report in Fig. 7, which shows a precision of 0.88

Classification Report:				
	precision	recall	f1-score	support
Anomaly (0)	0.88	0.69	0.77	12833
Normal (1)	0.68	0.87	0.76	9711
accuracy			0.77	22544
macro avg	0.78	0.78	0.77	22544
weighted avg	0.79	0.77	0.77	22544

Fig. 7. Local Outlier Factor Classification Report

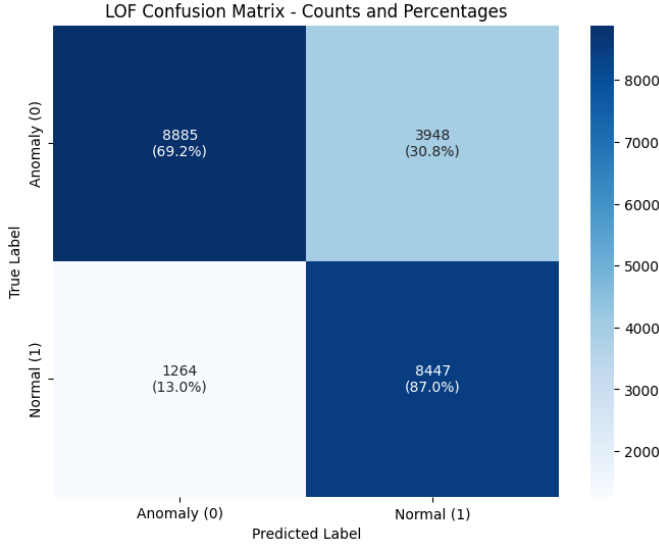


Fig. 8. Local Outlier Factor Confusion Matrix

and recall of 0.69 for anomalies, and a precision of 0.68 and a recall of 0.87 for normal data. These values suggest that the model is slightly better at correctly identifying normal samples than it is at consistently identifying anomalies. However, the relatively high precision for anomalies indicates that when the model does predict an anomaly, it is usually correct. The F1-scores of approximately 0.77 for both classes confirm that the model maintains a balanced trade off between precision and recall across both categories.

The confusion matrix in Fig. 8 provides a visual representation of these results. As shown, the model correctly predicted 69.2% of the anomalous data and 87% of normal samples. The amount of normal positive data that was high suggests that the algorithm's local density estimation is more aligned with the characteristics of the normal class.

To further assess the prediction capability, a ROC curve was generated and can be seen in Fig. 9. The curve shows an AUC of 0.792. This indicates that the LOF model possesses a moderate to strong ability to differentiate between normal and anomalous samples. This AUC suggest that in nearly 79% of random chosen cases, the model assigns a higher anomaly score to an actual anomaly than to normal data points. This confirms that while LOF captures some significant structural differences in local density between the two classes, some overlap remains.

During implementation of the new preprocessing, we en-

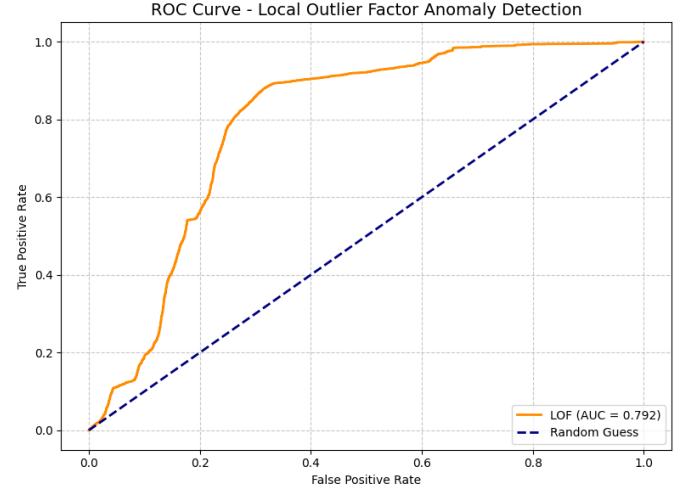


Fig. 9. Local Outlier Factor ROC Curve

countered persistent errors while trying to migrate, so the modifications were never successfully applied to the LOF model. Because we implemented this algorithm fairly early into our project, and because we were not able to get a higher success rate in classification, we ultimately abandoned LOF in favor of alternative algorithms that would likely have a higher success rate.

C. Autoencoder

Our autoencoder had low metrics. It had 72% accuracy, 65% precision, 75% recall, and an F1 score of 69%. Our auc value can be seen in Fig. 11. The autoencoder's confusion matrix can be seen in Fig. 12. 69.4% of normal data was correctly classified as normal, 30.6% of normal data was incorrectly classified as anomalous, 75.5% of anomalous data was correctly classified as anomalous, and 24.5% of anomalous data was incorrectly classified as normal. It seems that our autoencoder has a more difficult time correctly classifying normal data compared to anomalous data. Our loss during our training can be seen in Fig. 10. The training loss and validation loss drop to almost 0 by the end of training.

D. XGBoost

In Fig.10, we can see the loss and validation loss from training our autoencoder model. Both the validation loss and loss decreased over time to around 0.01. Both decrease to the same amount by the end of training our model.

In Fig.12 is our confusion matrix showing the results of testing our model. Our model correctly identifies 39.51% of our data as negative. Our autoencoder model correctly identifies 32.53% of our data as positive. The model incorrectly classifies 10.55% of our data as negative. The model incorrectly classifies 17.41% of our data as positive.

For XGBoost, using our current setup, we were able to achieve perfect accuracy. Looking at the current classification report in Fig. 13, we can see that we have a precision, recall, and F1 score of 1.0.

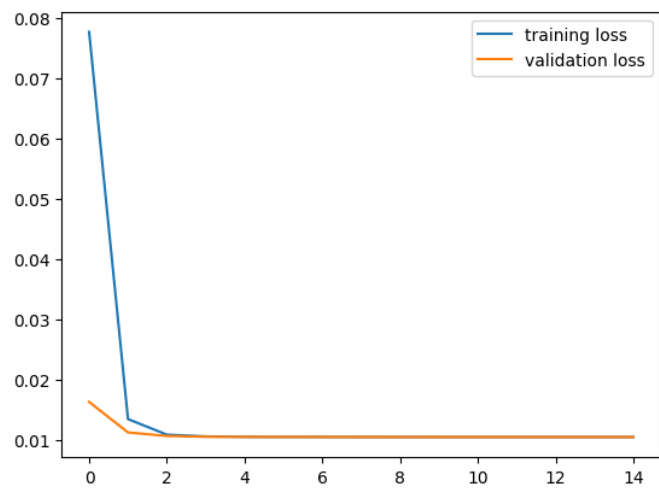


Fig. 10. Autoencoder Loss Diagram

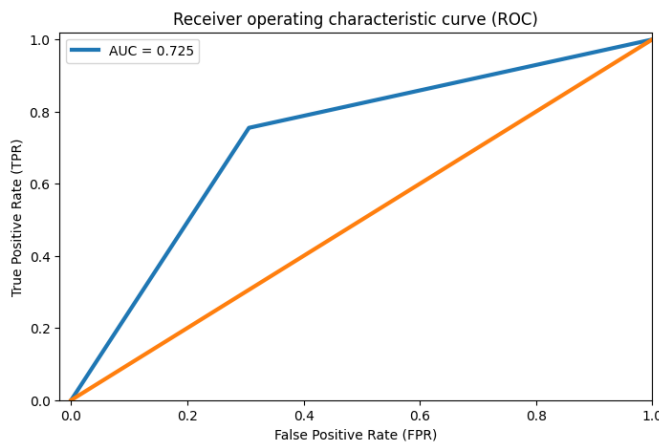


Fig. 11. Autoencoder ROC Curve

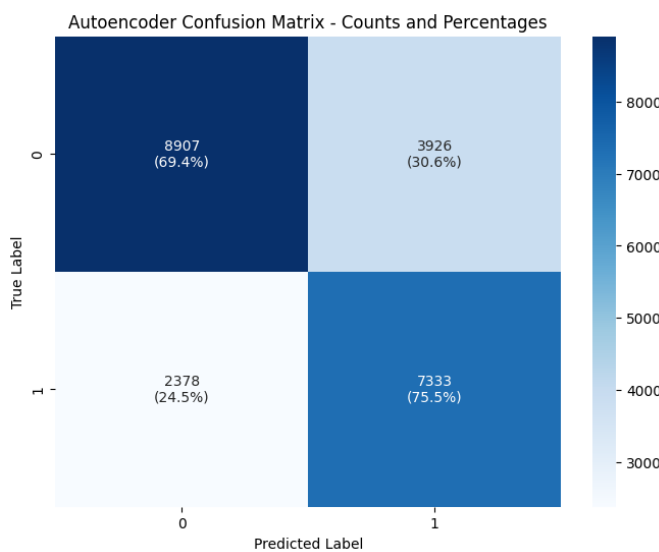


Fig. 12. Autoencoder Confusion Matrix

Classification Report:

	precision	recall	f1-score	support
Anomaly (0)	1.00	1.00	1.00	6733
Normal (1)	1.00	1.00	1.00	37823
accuracy			1.00	44556
macro avg	1.00	1.00	1.00	44556
weighted avg	1.00	1.00	1.00	44556

Fig. 13. XGBoost Classification Report

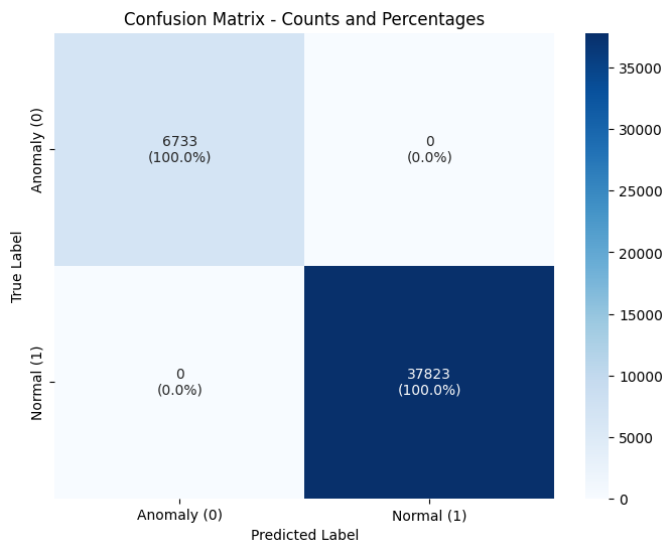


Fig. 14. XGBoost Confusion Matrix

Looking at our confusion matrix in Fig.14, we can see that XGBoost was 100% able to correctly differentiate between anomalous and normal network traffic. This makes XGBoost the most accurate model we tested.

None of this is surprising, as XGBoost is a supervised learning algorithm. This means that we give XGBoost labeled

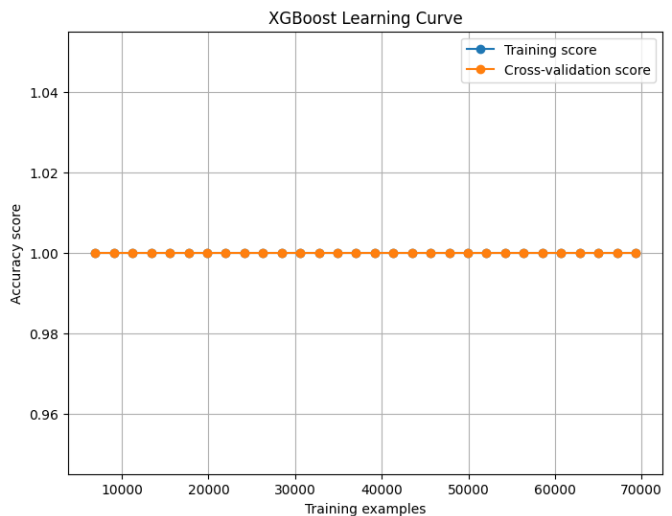


Fig. 15. XGBoost Learning Curve

data, and it quickly learns how to classify anomalies. Because the model is supervised, it is easy for overfitting to occur. However, after the preprocessing made the data more normalized, we were able to achieve a well-fit algorithm, as seen in Fig. 15. This means that XGBoost can both accurately capture the patterns in the training data, and also perform well on new, unseen data.

V. DISCUSSION

A. Overall Comparison

XGBoost achieved the highest performance metrics. It reached 100 percent Accuracy, Precision, Recall, F1Score, and Area Under the Curve. These results indicate that the model classified all samples correctly and maintained complete consistency across every aspect of its predictive performance. The isolation Forest model also demonstrated strong performance, with an Accuracy of 91%, Precision of 92%, Recall of 86%, F1 Score of 89%, AUC 90%. Although the Isolation forest generally performed well, it showed reduced recall compared to its precision, which suggest a higher tendency to miss certain anomalies.

The Autoencoder performed notably worse than other models, achieving 72% Accuracy, 65% Precision, 75% Recall, a 69% F1 Score, and an AUC of 73%. While the Autoencoder showed a higher sensitivity than precision, meaning it detected a larger portion of anomalies but generated more false positives, its overall performance remained much lower than other approaches. This outcome shows the models limitations in scenarios requiring high-confidence anomaly detection.

Overall, XGBoost demonstrated the best predictive performance, while the Isolation Forest offered a strong traditional baseline, and the Autoencoder provided moderate results but likely requires additional tuning to reach comparable performance metrics.

B. Comparison to Similar Models

Our isolation forest and XGBoost models managed to achieve good results for anomalous network traffic. When compared to the models used in [2], the isolation forest and XGBoost overall achieved better metrics over the models used. The only exception is that the K nearest neighbors model achieved better metrics than isolation forest with all metrics around 97% and 98%. The isolation forest used in [3] had better metrics than our isolation forest. The autoencoder reached higher metrics close to 100%. The autoencoder did not perform than any other model due as it had extremely low metrics.

C. Limitations

The accuracy and other metrics of the isolation forest could still be improved. The parameters could be tuned in order to achieve a better result or preprocessing could be altered in order to achieve better results. The setup of the model could be altered in order to improve the model's metrics. For example, a decision function could be applied to the model in order to increase accuracy, recall, etc.

A limitation for XGBoost is that it combines the original training and testing datasets before performing a new stratified split. While this approach helps balance class distributions and simplifies experimentation, it slightly reduces the strict separation between training and testing data. This means that the model evaluation results might be slightly better than they would be in a completely independent test scenario. However, the method remains useful for exploratory analysis and model development, as it provides a consistent and balanced dataset for overall model performance.

Another limitation is, because NSL-KDD used traffic from the 1990s, it does not actually include newer types of attacks. Something about not being good with zero-days, something about us needing to use a newer dataset.

D. Reproducibility

For this paper, we used the NSL-KDD dataset. This dataset is publicly available online to download and use. We are also making our code publicly available on GitHub. Our source code for all of our models that we discussed in this paper can be found at <https://github.com/sm-sydney/CYBR-495-Cyber-Threat-Detection-via-MachineLearning>.

VI. CONCLUSION

REFERENCES

- [1] D. U. M. N. Kavitha S and Dr.R.Venkatesh, "Network anomaly detection for nsl-kdd dataset using deep learning," *IT in Industry (Volume: 3, No. 2)*, 2021.
- [2] A. D. Vibhute, C. H. Patil, A. V. Mane, and K. V. Kale, "Towards detection of network anomalies using machine learning algorithms on the nsl-kdd benchmark datasets," *Procedia Computer Science*, vol. 233, pp. 960–969, 2024, 5th International Conference on Innovative Data Communication Technologies and Application (ICIDCA 2024). [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050924006458>
- [3] M. Vikram Aditya, "Anomaly detection in network traffic using unsupervised machine learning approach," 06 2020, pp. 476–479.